SchemeSathi: Automated Scholarship Eligibility Navigator

Comprehensive Java OOP Architecture, Enterprise File Handling & Faculty Defense Presentation

PRESENTER 1 (Lead Architect) **Priyanshu Gupta**	PRESENTER 2 (Backend & File I/O) **Team Member 2**	PRESENTER 3 (UI/UX & Integration) **Team Member 3**
• Project Vision & Problem Statement • 4 Fundamental OOP Pillars Mapping • Abstraction & Polymorphic Rules • Domain Model Encapsulation	• **Special Focus: Java File Handling** • NIO SchemeLoader JSON Ingestion • Dossier Export (ReportWriter) • CSV Audit Streaming & HTTP Server	• React 19 Frontend UI/UX Architecture • Dark/Light Theme System Design • End-to-End System Integration • 24 JUnit 5 Tests & Faculty Q&A;
Allotment: Slides 1 – 6 (5 Mins)	**Allotment: Slides 7 – 9 (5 Mins)**	**Allotment: Slides 10 – 12 (5 Mins)**

Target Faculty Audience: Object-Oriented Software Evaluation Committee **Total Time:** 15 Minutes (5 Mins/Member + Viva Q&A;) **Format:** Ready-to-Run Presentation Slide Deck

Project Overview & Problem Statement

Automated Scholarship Discovery & Verification Engine for Higher Education

The Real-World Academic Challenge	The SchemeSathi Solution
• **Fragmented Scholarship Landscape:** Indian students miss over 65% of available central, state, and private schemes due to opaque guidelines.	• **Automated Evaluation Engine:** Single consolidated interface taking student credentials and polymorphically evaluating every scheme in milliseconds.
• **Complex Multi-Variable Criteria:** Eligibility hinges on interconnected conditions: Academic Marks (%), Annual Family Income (■), and Social/Demographic Categories (General, OBC, SC, ST, EWS).	• **Transparent Reasoning & Checklists:** Outputs exact eligibility status, specific disqualification reasons, and personalized required document checklists.
• **High Manual Error Rate:** Students spend weeks filing invalid applications only to be rejected due to missing mandatory documents or income limits.	• **Multi-Channel Delivery:** Fully accessible as an interactive Terminal CLI (per PRD) and an enterprise React 19 Web SPA with embedded REST backend.

Pure Java OOP Core Abstraction, Encapsulation, Inheritance, Polymorphism	**Dual File Handling** NIO JSON Ingestion & Text Report Dossier Generation	**Enterprise Security** SHA-256 + 16-byte random salt authentication & CSV audit	**100% Test Verified** 24/24 JUnit 5 tests passing in continuous integration

OOP Pillar 1: Abstraction & Contract-Driven Design

Hiding Internal Implementation Details Behind Abstract Blueprints and Interfaces

1. Abstract Class: Scheme.java

• **Architectural Purpose:** Defines the root contract for all scholarship types. It encapsulates common fields (id, name, type, requiredDocuments) while deferring rule execution to subclasses.

• **Mandated Abstract Methods:**

```
public abstract boolean isEligible(Student student);
public abstract String getCriteriaSummary();
```

• **Why Abstract Class vs. Interface?**
All schemes share persistent state (IDs, Names, Document lists) and concrete getters/toString. Java interfaces cannot hold instance variables. An abstract class enables **state reuse + strict polymorphism**.

2. Interface: ReportWriter.java

• **Architectural Purpose:** Decouples the reporting subsystem from concrete storage devices. The business logic only knows about the contract, not file systems.

• **Interface Definition:**

```
public interface ReportWriter {
  void write(Path dest, Student s, List<Scheme> sc)
      throws IOException;
}
```

• **Extensibility Benefit:** SchemeSathi currently implements EligibilityReportWriter (text file), but can seamlessly swap to PdfReportWriter or DatabaseReportWriter without modifying calling code.

Key Faculty Takeaway: Abstraction in SchemeSathi establishes clean boundaries. The evaluation service never inspects concrete scheme internals; it simply invokes the abstract isEligible() contract.

OOP Pillar 2: Encapsulation & Defensive Security

Data Hiding, Class Invariants, and Cryptographic Security Bounds

Domain Invariant Encapsulation (Student.java)

• **Strict private Visibility:** All mutable instance variables (marksPercentage, annualFamilyIncome, category) are private.

• **Constructor Validation (Defensive Programming):**
Marks must be within [0.0, 100.0].
Income must be >= 0.0 (no negative financial figures).
Social category cannot be null or blank.

• **Immutability Principle:** State modifications throw explicit IllegalArgumentException before any corrupted object can enter the system runtime.

Cryptographic Security Encapsulation (AuthService.java)

• **Zero Plaintext Password Exposure:** Passwords are never stored in memory or data files in plaintext.

• **SHA-256 + 16-Byte Cryptographic Salt:**
Generates a unique 16-byte random salt per user using java.security.SecureRandom.
Hashes password + salt via MessageDigest.getInstance("SHA-256").
Prevents rainbow table attacks and dictionary exploits.

• **Session Token Isolation:** Generates cryptographically isolated Bearer tokens (STOK-...) stored in concurrent thread-safe maps.

```
// AuthService.java - Cryptographically Encapsulated Hashing private String hashPassword(String password, String salt) { MessageDigest md =
MessageDigest.getInstance("SHA-256"); md.update(salt.getBytes(StandardCharsets.UTF_8)); return HexFormat.of().formatHex(md.digest(password.getBytes(StandardCharsets.UTF_8)));
}
```

OOP Pillar 3: Inheritance & Subclass Specialization

Code Reusability, Constructor Chaining via super(), and Domain Hierarchy

Class Name	Parent & Inheritance Chain	Specialized Attributes	Eligibility Rule Implementation
Scheme (Abstract Base)	`java.lang.Object`	`id, name, type, requiredDocuments`	Defines abstract isEligible(Student) blueprint.
MeritScheme	`extends Scheme`	`double minimumMarks`	Evaluates: student.getMarks() >= this.minimumMarks
IncomeScheme	`extends Scheme`	`double maximumAnnualIncome`	Evaluates: student.getAnnualFamilyIncome() <= this.maxIncome
CategoryScheme	`extends Scheme`	`List<String> allowedCategories`	Evaluates: allowedCategories.contains(student.getCategory())

Constructor Chaining with super(...):

```
public MeritScheme(String id, String name, List<String> docs, double minMarks) {
super(id, name, "MERIT", docs); // Reuses parent constructor logic this.minimumMarks =
minMarks; }
```

Eliminating Redundancy:

- Avoids repeating field assignments, null checks, and defensive copies across each subclass.
- Guarantees that any state invariant enforced by Scheme is universally applied to all derived scholarships.
- Keeps subclass code strictly focused on criteria-specific business logic.

OOP Pillar 4: Polymorphism & Dynamic Method Dispatch

Runtime Binding and the Open/Closed Principle (OCP)

Source Code: Dynamic Method Dispatch in Action

```
// EligibilityService.java — Polymorphic Evaluation Engine public List
findEligible(Student student, List schemes) { List eligible = new ArrayList<>(); for
(Scheme scheme : schemes) { // RUNTIME POLYMORPHISM (Dynamic Method Dispatch): // JVM
inspects the exact object instance at runtime // and dispatches to MeritScheme,
IncomeScheme, or CategoryScheme. if (scheme.isEligible(student)) { eligible.add(scheme); }
} return eligible; }
```

Architectural & Academic Significance

• **What is Dynamic Dispatch?**
The compile-time reference is generic (Scheme), but at runtime the Java Virtual Machine dynamically resolves the virtual method table (vtable) to execute the subclass's overridden isEligible() method.

• **Adherence to Open/Closed Principle (OCP):**
Software entities should be *open for extension, closed for modification*. Adding a brand-new SportsQuotaScheme or DisabilityScheme requires **zero edits** to EligibilityService!

• **Zero Code Smells:**
Eliminates clumsy if (s instanceof MeritScheme) or type-casting ladders.

Handoff to Presenter 2: ***"Now, my colleague will delve into the critical technical centerpiece of our project — Java File Handling Architecture, JSON Ingestion, and CSV Streaming."***

Special Focus: Java File Handling Architecture (Input)

Robust File I/O, Defensive Path Validation, and JSON Schema Deserialization

Java NIO SchemeLoader.java Implementation

```
// SchemeLoader.java — Defensive Java NIO File Reading public List load(Path path)
throws IOException { if (path == null) throw new IllegalArgumentException("Path cannot
be null."); if (!Files.exists(path)) throw new IOException("Rules file does not exist: "
+ path.toAbsolutePath()); if (!Files.isRegularFile(path)) throw new IOException("Rules
file is not a regular file: " + path.toAbsolutePath()); if (!Files.isReadable(path))
throw new IOException("Rules file is not readable: " + path.toAbsolutePath()); // UTF-8
NIO Stream Ingestion String jsonContent = Files.readString(path,
StandardCharsets.UTF_8).trim(); if (jsonContent.isEmpty()) throw new IOException("Rules
file is empty"); return parseJson(jsonContent); }
```

Defensive File-Handling Principles Applied

• **Modern Java NIO.2 API (java.nio.file):**
Uses immutable Path and Files utility methods instead of legacy java.io.File for cross-platform file system compatibility.

• **3-Tier Pre-Flight File Checks:**
1. Files.exists(): Checks file presence.
2. Files.isRegularFile(): Protects against directory paths or corrupt symlinks.
3. Files.isReadable(): Prevents OS permission denials.

• **Explicit Character Encoding (UTF-8):**
Mandates StandardCharsets.UTF_8 to eliminate platform-dependent encoding bugs on Windows vs. Linux servers.

• **Schema Validation & Duplicate Detection:**
Deserializes into intermediate SchemeRule records, rejects malformed schemas, and verifies uniqueness via Set<String> seenIds.

Academic Defense Note: Unlike naive projects that crash with unhandled FileNotFoundException, SchemeSathi defensively catches, wraps, and reports actionable error diagnostics.

Special Focus: Java File Handling (Dossiers & CSV Export)

Deterministic File Writing, Directory Creation, and RFC-4180 CSV Streaming

1. Audit Report Writing (EligibilityReportWriter)	2. RFC-4180 CSV Audit Streaming (AuthService)
`// EligibilityReportWriter.java — Safe Disk Dossier Writing Path parent = destination.getParent(); if (parent != null && !Files.exists(parent)) { Files.createDirectories(parent); // Auto-creates 'output/' dir } Files.writeString(destination, report.toString(), StandardCharsets.UTF_8, StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING, StandardOpenOption.WRITE);`	`// AuthService.java — RFC-4180 Compliant CSV Generation StringBuilder sb = new StringBuilder(); sb.append('\ufeff'); // UTF-8 BOM for Microsoft Excel! sb.append("Log ID,Username,Email,Role,Timestamp,IP,Status\r\n"); for (LoginRecord r : records) { sb.append(escapeCsv(r.getId())).append(',') .append(escapeCsv(r.getUsername())).append(',') .append(escapeCsv(r.getStatus())).append("\r\n"); } return sb.toString();`
• **Automated Directory Provisioning:** Invokes Files.createDirectories(parent) so the app never throws NoSuchFileException on fresh machines. • **Atomic StandardOpenOptions:** Uses CREATE, TRUNCATE_EXISTING, and WRITE to ensure clean overwrites without stale byte remnants.	• **Excel Compatibility:** Prepends UTF-8 Byte Order Mark (\ufeff) to prevent character corruption when opened in Microsoft Excel. • **CSV Injection & Quoting Escaping:** Custom escapeCsv() wraps commas and doubles quotation marks ("") per RFC-4180.

Backend Architecture & REST Microserver Layer

Lightweight Embedded HttpServer, Clean Separation of Concerns, and Endpoints

Layer / Package	Primary Classes	Responsibilities & Engineering Value
Domain Model	`Student, Scheme, MeritScheme, IncomeScheme, CategoryScheme`	Core business models, inheritance hierarchies, and polymorphic contracts.
Service Layer	`EligibilityService`	Polymorphic scheme matching and disqualification criteria breakdowns.
Persistence & I/O	`SchemeLoader, ReportWriter, EligibilityReportWriter`	Java NIO JSON ingestion, rule parsing, and formatted dossier file output.
Security & Auth	`AuthService, User, LoginRecord`	SHA-256 + 16B salt hashing, session token store, and CSV audit logs.
REST Server	`ApiServer, EvaluateHandler, ReportHandler, AdminCsvHandler`	Embedded Java HttpServer on port 8080 with CORS headers & JSON responses.

Handoff to Presenter 3: ***"Next, my colleague will demonstrate the modern React 19 Frontend UI/UX, system integration, automated test verification, and faculty viva preparation."***

Frontend UI/UX Design System & Interface

Modern React 19 SPA, Clean Typography, Dual Themes, and Admin Portal

Modern UI/UX Ergonomics	Enterprise Admin & System Features
• **Dual Theme Design System (Dark & Light Mode):** Engineered with semantic CSS Custom Properties (--bg-main, --text-primary, --accent-cyan) for instant, seamless theme transitions. • **1-Click Evaluation Demographic Presets:** Interactive presets allow instant evaluation of realistic student archetypes (*High Merit 92%, Need-Based ■2.2L, OBC Candidate, SC/ST Special, EWS Scholar*). • **Clean, Uncluttered Visual Focus:** Refined layout with background animation removed, ensuring maximum visual accessibility, fast render performance, and zero cognitive distraction for evaluators.	• **Dedicated Administrator Portal (/admin):** Isolated administrative route with real-time audit logs, tracking login timestamps, user roles, IP addresses, and authentication outcomes. • **1-Click CSV Audit Trail Download:** Directly streams the generated CSV file from the Java backend for institutional compliance and record keeping. • **Instant Dossier Generation:** Students can click *'Download Audit Dossier'* to trigger ReportHandler and download the formatted plain-text report onto their local machine.

Team Work Division & Defense Presentation Plan

Clear Responsibility Mapping, Presentation Timing, and Speaker Hand-Offs

Member & Role	Assigned Presentation Scope	Key Slide Range & Speaking Time	Defense Hand-Off Responsibilities
Member 1: Priyanshu Gupta *System Architect & OOP Core Lead*	• Project Vision & Problem Solved • 4 Fundamental OOP Pillars (Abstraction, Encapsulation, Inheritance, Polymorphism) • Dynamic Method Dispatch Proof	**Slides 1 – 6** 0:00 – 5:00 min (5 Mins)	Sets up architectural context; introduces domain problem and passes baton to Member 2.
Member 2: Team Member 2 *Backend & File-Handling Lead*	• **Special Focus: Java File Handling** • NIO SchemeLoader JSON Ingestion • Dossier Report Writing to Disk • CSV Log Streaming & REST Microserver	**Slides 7 – 9** 5:00 – 10:00 min (5 Mins)	Explains file I/O safety, error handling, try-with-resources; passes baton to Member 3.
Member 3: Team Member 3 *UI/UX & Integration Lead*	• React 19 Frontend UI/UX Design • Dual-Theme Accessibility System • 24 JUnit 5 Automated Tests • Cloud Deployment & Viva Defense	**Slides 10 – 12** 10:00 – 15:00 min (5 Mins)	Demonstrates end-to-end integration and opens the floor for faculty viva questions.

Verification Suite & Faculty Viva Defense Q&A;

24 JUnit 5 Automated Tests & Top Faculty Defense Questions with Model Answers

Automated Test Suite Status: ✓ 24 / 24 Tests Passing (100% Success) | StudentTest (9) • SchemeTest (3) • EligibilityServiceTest (2) • SchemeLoaderTest (3) • EligibilityReportWriterTest (2) • AuthServiceTest (5)

Faculty Viva Question	Bulletproof Model Answer (Defended by Assigned Member)
Q1: Why use an abstract class instead of an interface for Scheme?	*(Presenter 1)* All schemes share common state variables (id, name, requiredDocuments) and shared getters. Interfaces cannot hold instance state. An abstract class maximizes code reuse while guaranteeing mandatory polymorphic behavior via abstract isEligible().
Q2: Where exactly does dynamic method dispatch occur?	*(Presenter 1)* In EligibilityService.findEligible() inside the loop scheme.isEligible(student). The JVM resolves the concrete implementation (MeritScheme, IncomeScheme, or CategoryScheme) at runtime via the vtable.
Q3: How does file handling prevent application crashes on invalid input?	*(Presenter 2)* SchemeLoader performs 3-tier pre-flight checks (Files.exists, isRegularFile, isReadable) and wraps parsing exceptions into detailed IOException. EligibilityReportWriter auto-creates missing parent directories.
Q4: How are passwords protected against dictionary and rainbow table attacks?	*(Presenter 2)* We use java.security.SecureRandom to generate a cryptographically secure 16-byte random salt per user. Passwords are never stored in plaintext; only the SHA-256 hashed digest is persisted.
Q5: How does the client communicate with the Java server?	*(Presenter 3)* Through an embedded lightweight Java HttpServer listening on port 8080. It exposes REST endpoints with CORS headers (Access-Control-Allow-Origin: *) exchanging JSON payloads with the React 19 SPA.